

Alfido Tech DevOps Internship

ML Inference API — Incident Response Runbook

Intern: KVSajith34 | Version: 1.0 | Date: 19 May 2026

1. Service Information

Property	Value
Service Name	ML Inference API
Model	Iris Classifier — RandomForestClassifier
Language	Python 3.11 / Flask
Port	5000
Health Endpoint	http://localhost:5000/health
Metrics Endpoint	http://localhost:5000/metrics
Prometheus	http://localhost:9090
Grafana Dashboard	http://localhost:3000
Owner	KVSajith34 — Alfido Tech DevOps Intern

2. Alert Severity Levels

Severity	Response Time	Action Required
■ Critical	Immediate (< 5 min)	Page on-call, begin incident response
■ Warning	15 minutes	Investigate, monitor for escalation
■ Info	Next business day	Log and review during working hours

3. Incident 1 — ML API Service Down

■ CRITICAL | Alert: MLAPIDown

Detection

■ Prometheus alert fires: up{job='ml-inference-api'} == 0 for 30 seconds

Step	Action	Command
1	Check container status	<code>docker ps grep ml-api</code>
2	Check container logs	<code>docker logs ml-api --tail 50</code>
3	Check health endpoint	<code>curl http://localhost:5000/health</code>
4	Check Prometheus target	<code>http://localhost:9090/targets</code>

Mitigation

Step	Action	Command
------	--------	---------

1	Restart the container	<code>docker restart ml-api</code>
2	Verify restart	<code>docker ps grep ml-api</code>
3	Test health endpoint	<code>curl http://localhost:5000/health</code>
4	Send test prediction	<code>curl -X POST http://localhost:5000/predict -H "Content-Type: application/json" -d '{"features":[5.</code>
5	Check Prometheus target	Verify UP at localhost:9090/targets

Post-Mortem

Item	Action
Root Cause	Identify why container stopped (OOM, crash, config error)
Timeline	Document time of alert, response, and resolution
Prevention	Add health check restart policy in docker-compose
Follow-up	Review logs for recurring errors, add memory limits

Resolution Verification

■ Service restored when: health endpoint returns 200, Prometheus shows UP, predictions return valid class with confidence > 0.7

4. Incident 2 — High Prediction Error Rate

■ **WARNING | Alert: HighErrorRate**

Detection

■■ Prometheus alert fires: `rate(ml_error_total[5m]) > 0.1` for 1 minute

Step	Action	Command
1	Check error metrics	<code>curl http://localhost:5000/metrics grep ml_error</code>
2	Check API logs	<code>docker logs ml-api --tail 100 grep ERROR</code>
3	Check error types	Look for <code>validation_error</code> vs <code>server_error</code> in logs
4	Test predict endpoint	<code>curl -X POST localhost:5000/predict -d '{"features": [5.1, 3.5, 1.4, 0.2]}'</code>

Mitigation

Error Type	Cause	Fix
<code>validation_error</code>	Bad input format from client	Fix client payload — must be 4 features
<code>server_error</code>	Model file missing or corrupt	Rebuild container: <code>docker compose up --build</code>
500 responses	Python exception in <code>app.py</code>	Check logs, fix code, redeploy

Post-Mortem

Item	Action
Root Cause	Determine if client-side or server-side error
Fix	Patch input validation or fix model loading
Prevention	Add input schema validation (e.g. <code>marshmallow</code> / <code>pydantic</code>)
Follow-up	Add error rate dashboard alert threshold panel in Grafana

5. Incident 3 — High Request Latency

■ **WARNING | Alert: HighRequestLatency (P95 > 500ms)**

Detection

■■ Prometheus alert fires: `histogram_quantile(0.95, rate(ml_request_latency_seconds_bucket[5m])) > 0.5`

Step	Action	Command
1	Check latency metrics	<code>curl localhost:5000/metrics grep latency</code>
2	Check system load	<code>docker stats ml-api</code>
3	Check Grafana P95 panel	<code>localhost:3000</code> → P95 Latency panel
4	Check active requests	<code>curl localhost:5000/metrics grep active_requests</code>

Mitigation

Cause	Fix
High traffic volume	Scale unicorn workers: increase <code>--workers</code> in Dockerfile

Memory pressure	Increase container memory limit in docker-compose
Model too slow	Optimize model: reduce n_estimators, use joblib caching
System CPU spike	Check node-exporter metrics in Grafana for host load

Post-Mortem

Item	Action
Root Cause	Identify if traffic spike, resource exhaustion, or model issue
Prevention	Set gunicorn timeout, add rate limiting, pre-load model in memory
Follow-up	Load test with locust/k6, optimize inference pipeline

6. Incident 4 — Low Model Prediction Confidence

■ **WARNING | Alert: LowPredictionConfidence (< 0.7)**

Detection

■ Prometheus alert fires: ml_prediction_confidence < 0.7 for 1 minute — possible data drift or out-of-distribution inputs detected.

Step	Action	Command
1	Check confidence gauge	curl localhost:5000/metrics grep confidence
2	Check prediction distribution	Grafana → Predictions by Class (should be balanced)
3	Log sample inputs	docker logs ml-api grep prediction tail 20
4	Test with known samples	curl -X POST localhost:5000/predict -d '{"features":[5.1,3.5,1.4,0.2]}'

Mitigation

Cause	Fix
Out-of-distribution input	Validate input ranges before prediction
Model data drift	Retrain model with recent data, redeploy container
Wrong feature scaling	Check if input preprocessing matches training data

Post-Mortem

Item	Action
Root Cause	Compare production input distribution vs training data
Prevention	Add input range validation, monitor confidence trend over time
Follow-up	Implement model retraining pipeline, add confidence histogram panel

7. Quick Reference — Emergency Commands

Scenario	Command
Restart all services	<code>docker compose restart</code>
Rebuild + restart	<code>docker compose up --build -d</code>
Check all logs	<code>docker compose logs --tail 50</code>
Check ML API logs	<code>docker logs ml-api --tail 50</code>
Check metrics	<code>curl http://localhost:5000/metrics grep ml_</code>
Test health	<code>curl http://localhost:5000/health</code>
Test prediction	<code>curl -X POST http://localhost:5000/predict -H "Content-Type: application/json" -d '{"features": [5.</code>
Stop all services	<code>docker compose down</code>
View Prometheus	<code>http://localhost:9090</code>
View Grafana	<code>http://localhost:3000 (admin/alfido123)</code>